

東海大學資訊工程學系學士班

學士專題

無參數更新之技能層級演化
——大型語言模型代理的結構化記憶形式框架

**Skill-Level Evolution Without Parameter Updates:
A Formal Framework for Structured Memory in LLM Agents**

專題學生：黃柏璋、鍾庭立

指導教授：楊朝棟教授

中華民國 115 年 9 月

(一) 摘要

本研究提出自我演化代理系統（以下簡稱 SIES），探討在不更新模型參數的前提下，使大型語言模型代理透過結構化記憶中技能的累積、精煉與重組而持續成長。我們將代理形式化為四元組 $A = (M, \Sigma, R, \Phi)$ ，以三模組推理機制（任務分解 A_1 、記憶引導規劃 A_2 、反思評估 A_3 ）構成「執行—反思—入庫—再利用」的閉環，並以演化算子 Φ 對加權技能圖譜執行效用評估、插入、子圖收縮與三層分級，輔以課程式自動出題形成無人監督的持續學習迴圈。在僅變動 A_3 反思策略的 50 題控制實驗中，改良版將候選技能產出降低 52%、圖譜插入降低 45%。另以同一批凍結題庫進行停用技能庫的對照：兩組獨立驗證通過率同為 32/43（74.4%），但逐題一致率僅 76.7%，顯示在此規模下，技能庫對通過率的影響小於語言模型本身的隨機性；停用技能庫則使提示詞 token 減少 19.8%。SIES 的演化過程外顯於可讀的技能檔案，具可解釋性與可回溯性，其演化性質以一組形式命題刻畫，多數尚待實證；本文並逐項載明實作與形式框架的落差（如 216 條邊中由系統學得者為 0 條），作為後續可分辨實驗設計的依據。

(二) 研究動機與研究問題

2.1 研究動機

大型語言模型（LLM）近年在推理、工具使用與多步驟任務執行上展現強大能力，然而現有的 LLM 代理多屬結構靜態設計：代理在不同任務之間無法累積可重用的程序性知識（procedural knowledge），每次面對新任務皆近乎從零開始推理，難以將過往經驗沉澱為長期能力。

若要使代理具備持續適應能力，常見做法是透過參數微調（fine-tuning）或強化學習修改模型權重。然而此類方法面臨數項根本限制：訓練成本高昂、易發生災難性遺忘（catastrophic forgetting）、權重更新不可逆，且演化過程封裝於數十億參數之中而難以解釋與審計。對於需要長期累積、逐步成長的代理而言，這些限制使其能力演化既不透明也不易回溯。

本研究的動機在於探索一條不同的技術路線：**在不修改模型參數的前提下，讓代理透過外部記憶中「技能」的累積、精煉與重組，實現能力的持續成長。**我們將代理的經驗組織為可檢索、可更新的結構化技能單元，透過「執行任務 → 記錄軌跡 → 反思萃取 → 技能入庫 → 下次任務受益」的迭代閉環，使能力的演化發生在外顯且可讀的技能層級。此路線具三項實務價值：(1) 演化過程完全基於外顯化的技能檔案與圖譜紀錄，可解釋、可回溯、可審計；(2) 模型權重凍結，從根本避免災難性遺忘；(3) 系統能在同一凍結模型上持續累積跨任務經驗，使處理相似任務的推理步驟逐漸縮減。

2.2 研究問題

承上，本研究的核心問題可表述為：在模型參數固定的條件下，如何建構一套代理系統，使其能透過反覆的任務執行與反思，自主累積可重用的技能，並使這些技能隨時間推移持續改善任務處理能力？我們將其分解為三個研究問題：

RQ1（表徵與演化）：如何以結構化記憶（而非平坦列表）表徵技能及其關聯，並設計演化算子在不更新參數的前提下，自動管理技能的插入、收縮與分層？

RQ2（反思與萃取）：如何從代理的執行軌跡——特別是失敗或低效的軌跡——中，辨識並萃取出可抽象、可重用且通過驗證的技能？

RQ3（成效驗證）：技能層級的演化能否隨任務累積有效提升技能重用度、縮減推理步數，同時維持任務成功率？此成效應可被量化觀測與驗證。

此三個研究問題分別對應後續章節：RQ1 由第（四）節的代理四元組與演

化算子 Φ 回答，RQ2 由 A_3 反思評估器回答，RQ3 則由第（五）節的控制實驗與長期演化實驗檢驗。

（三）文獻回顧與探討

3.1 技能庫與程式碼重用

Voyager [5] 在 Minecraft 環境中提出技能庫 (skill library) 概念，將驗證成功的程式碼存為可重用技能並透過反思持續擴充，展現「以技能累積取代重複探索」的潛力；惟其技能為綁定特定環境的可執行程式碼，跨任務泛化與組織結構較受限。LATM [1] 則讓模型扮演「工具製造者」，自動生成並驗證可重用工具供後續任務呼叫；但完整的工具生成需配套型別檢查、安全審計與回歸測試，工程複雜度與風險偏高。兩者共同指出程式碼重用的價值，也凸顯其在驗證成本與可審計性上的取捨。

3.2 技能演化與記憶壓縮

SkillRL [6] 以 SkillBank 區分通用與任務專屬技能，並從失敗中遞迴演化技能，是與技能演化最接近的實作參考；SimpleMem [3] 則針對長期記憶提出語義壓縮、遞迴整合與查詢感知檢索三階段機制，可作為冷卻／封存層的壓縮引擎。此二者提供了技能與記憶層面的演化與壓縮策略，但多以經驗式流程呈現，缺乏對圖譜規模、效用分布等性質的形式化保證。

3.3 執行框架與自我挑戰

在執行層面，Agent-Zero [4] 提供原生支援 SKILL.md 標準、Docker 沙盒與子代理結構的通用代理框架，適合作為技能的「消費端」執行環境。在任務來源層面，課程式 (curriculum) 自我挑戰機制可依代理當前能力自動生成「前沿任務」，驅動其能力邊界之外的學習；近期通用代理研究 [2] 亦朝可重用能力與自我改進方向發展。

3.4 2026 年的技能圖檢索與技能庫策展

本研究進行期間，2026 年另有數條與 SIES 高度相鄰的研究線。在**技能圖檢索**方面，SkillTrace (arXiv:2608.02356) 以 PageRank 遍歷技能圖挑選技能，Graph of Skills (arXiv:2604.05333) 採圖上傳播，SkillDAG (arXiv:2606.03056) 以型別化的邊做 BFS，GraSP (arXiv:2604.17870) 則將技能組織為前置條件—效果的 DAG；在**技能庫策展**方面有 SkillBrew (arXiv:2605.29440)；在**自軌跡抽取技能**方面有 AutoSkill (arXiv:2603.01145) 與 EvoSkill (arXiv:2603.02766)；另有 SoK: Agentic Skills (arXiv:2602.20867) 嘗試對此類系統作形式化整理。SIES 與上述工作的差異有三：(a) 前述系統多在**檢索期**對既有技能作組合、排序或遍歷，圖本身不因查詢而改變，SIES 的 Φ -iii 則是對圖的**寫入操作**——永久合併出新節點並將 parent 降層，改變的是圖的結構而非單次查詢的結果；(b) SIES 的層級遷移由連續時間的效用衰減搭配四個閾值 (θ_{high} 、 θ_{low} 與兩個遲滯帶) 驅動，而非單一靜態的排序函數；(c) 推理、嵌入與演化全程在單機本地完成（語言模型於單張消費級 GPU、嵌入模型於 CPU），不倚賴任何外部 API。

3.5 本研究之定位

綜合而言，SIES 並不追求完整的工具自動生成（如 LATM），亦不停留於僅儲存策略文字的純技能演化，而是介於兩者之間：在技能層級實現程式碼與策略的重用，但不修改工具註冊表，從而保持演化過程的可解釋性與可審計性。更重要的是，SIES 將技能組織為帶有演化算子 Φ 的加權圖譜，並以一組形式化命題（圖譜規模有界、規劃深度下降、熵有界收斂、效用分布 Gini 非遞減）刻畫其演化性質——這是上述多數經驗式系統所缺乏的理論面向，亦為本研究的主要切入點。

(四) 研究方法及步驟

本研究以團隊先前提出之論文框架為理論依據，將前述 SIES 建構為一套可運行的原型。本節先界定代理的整體形式化結構，再依序說明三模組推理閉環、結構化技能圖譜與記憶分級、演化算子，以及自我挑戰的課程式閉環，最後交代系統實作堆疊與研究步驟。

4.1 整體架構：代理四元組

我們將代理形式化為一個四元組，作為全系統設計的理論骨架。

Definition 1 (自我演化代理). 一個自我演化代理定義為四元組 $A = (M, \Sigma, R, \Phi)$ ，其中 $M = (M_e, M_s, M_w)$ 為由情節記憶、結構化技能記憶與工作記憶構成的記憶系統； Σ 為技能集合； $R = A_1 \rightarrow A_2 \rightarrow A_3$ 為三模組推理機制； Φ 為作用於技能圖譜的演化算子。

此架構的關鍵特性在於：代理的「成長」完全發生在技能集合 Σ 及其組織結構上，而非模型參數 θ 。在凍結的語言模型之上，透過 Φ 對 Σ 的反覆作用，使代理能力隨任務經驗累積而提升；整個演化過程外顯於可讀的技能檔案與圖譜索引，具備可解釋性與可回溯性。整體資料流與演化閉環如圖 1 所示。

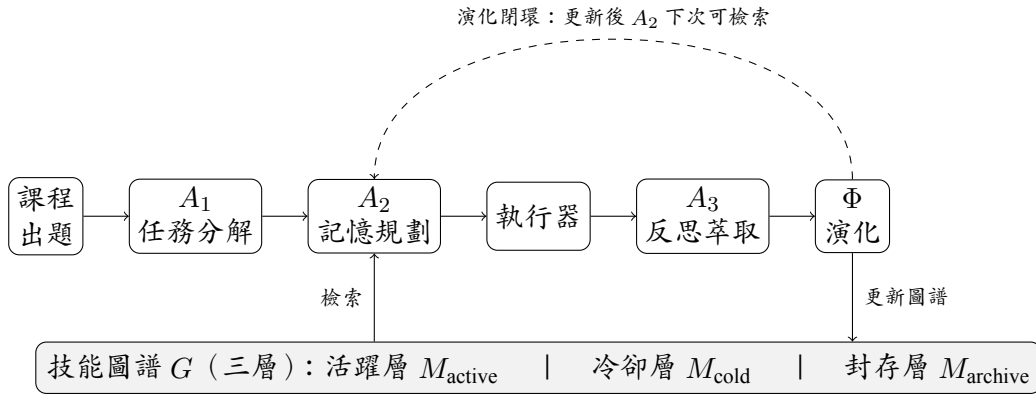


圖 1 SIES 整體資料流與演化閉環

4.2 三模組推理閉環 ($A_1 \rightarrow A_2 \rightarrow A_3$)

代理處理單一任務的推理流程由三個模組串接而成。 A_1 **任務分解器**將自然語言任務 T 結構化為 $T_{\text{struct}} = \{\text{subtasks, constraints, objectives}\}$ ，並標注子任務依賴、風險等級與預期產出。 A_2 **記憶引導規劃器**以 T_{struct} 為查詢，瀏覽技能圖譜並檢索最相關的 top- k 技能注入工作記憶，產生帶有來源標注的執行計畫；技能的選取依活化分數（見 4.3）排序。 A_3 **反思評估器**於任務執行後分析其軌跡 τ ，從成功或失敗的模式中萃取候選技能 $\Delta\Sigma$ ，送交演化算子評估。三模組之職責與輸入輸出整理如下表：

模組	功能	輸入 → 輸出
A_1 任務分解器	任務結構化	$T \rightarrow T_{\text{struct}}$
A_2 記憶引導規劃器	技能檢索與計畫生成	$T_{\text{struct}} + G \rightarrow \text{執行計畫}$
A_3 反思評估器	軌跡反思與技能萃取	$\tau \rightarrow \Delta\Sigma$

此 $A_1 \rightarrow A_2 \rightarrow A_3$ 流程構成「執行—反思—入庫—再利用」的閉環：本次任務萃取的技能，將在後續相似任務中被 A_2 檢索並重用，使推理步驟逐漸縮減。

4.3 結構化技能圖譜與三層記憶分級

不同於將技能組織為平坦列表，本系統以加權有向圖 $G = (\Sigma, E)$ 表示技能與其關聯，邊權重編碼技能間的組合與依賴關係。每個技能 σ 具有效用值，定義為

$$U(\sigma) = \alpha \cdot r + \beta \cdot f - \gamma_c \cdot c, \quad (1)$$

其中 r 為任務回饋、 f 為使用頻率、 c 為呼叫成本，並以 $U_{t+1} \leftarrow (1 - \gamma) U_t + \Delta U_t$ 進行時間衰減，使長期未用的技能自然降權。

依效用值高低，技能被分為三層：活躍層 M_{active} （每次任務優先載入工作記憶）、冷卻層 M_{cold} （可檢索但優先級低）與封存層 M_{archive} （永不刪除，但預設不檢索）。按設計， A_2 應僅檢索活躍層；現行實作尚未依層級過濾（見 5.5 節 (f)），工作記憶用量則由 top- k 截斷，與技能庫總量無關。三層分級整理如下表：

層級	效用條件	檢索行為（設計）
活躍層 M_{active}	$U(\sigma) \geq \theta_{\text{high}}$	每次任務優先載入
冷卻層 M_{cold}	$\theta_{\text{low}} < U(\sigma) < \theta_{\text{high}}$	可檢索、優先級低
封存層 M_{archive}	$U(\sigma) \leq \theta_{\text{low}}$	保留但預設不檢索

技能的檢索排序則依活化分數：

$$\text{Act}(\sigma) = \lambda_1 \cdot \text{sim} + \lambda_2 \cdot U(\sigma) + \lambda_3 \cdot \text{centrality}(\sigma), \quad (2)$$

綜合語義相似度、效用與圖中心性三項；其中語義相似度由向量檢索回傳的距離 d 換算而得（向量已正規化， $\text{sim} = 1 - d/2$ ）¹。

4.4 演化算子 Φ

Φ 是系統自我演化的核心，於每批次任務後作用於圖譜 $G_t \rightarrow G_{t+1}$ ，由四個子算子構成。 **Φ -i 效用評估**：依真實軌跡更新各技能的 $U(\sigma)$ 。 **Φ -ii 技能插入**：對 A_3 產出的候選技能執行三重驗證——格式檢查、去重（餘弦相似度 $> \theta_{\text{dup}} = 0.75$ 即視為重複而剔除）、品質檢查——通過者插入圖譜。 **Φ -iii 子圖收縮**：以共現統計（support 與 lift 雙閾值）找出高頻共同出現的技能對，合併為宏技能（macro-skill）節點，以壓縮規劃深度。 **Φ -iv 記憶分層**：依更新後的效用值，以遲滯（hysteresis）閾值將技能在三層之間遷移，避免邊界震盪。

關於 θ_{dup} 的取值：此處的 0.75 係在修正 4.3 節的相似度公式之後重新挑選。先前採用的 0.80 是在**修正前的錯誤度量**上選定的——該度量會系統性高估相似度，因此其門檻值不具可移植性，不能直接沿用到正確的度量上。公式修正、並為技能索引補上「執行策略」段（使嵌入向量能反映技能的實際做法而非僅標題）之後，我們以聚合式分群（agglomerative clustering，cosine 距離、average linkage）重新檢視技能庫：在餘弦相似度 ≥ 0.75 時，既有的三個真實同義技能族各自正確成群；門檻再放寬則開始把語意不同的族群併在一起。0.75 即取自此一邊界。

關於收縮語意的一項刻意偏離：論文框架的 Definition 7 將子圖收縮寫為 $\Sigma \leftarrow (\Sigma \setminus \{\sigma_i, \dots, \sigma_j\}) \cup \{\sigma^*\}$ ，即把被合併的來源技能（parent）自技能集合中**移除**。本實作**不移除** parent，而是將其保留於圖上、並把層級降為冷卻層。此偏離是刻意的：圖層保留 parent 作為巨集技能的來源證據（provenance）， σ^* 與其 parent 之間方能建立 composes_into 邊；若依原式移除，這些邊、以及「巨集技能是否與圖連通」這項驗收指標都將無從成立。語意上， σ^* 取代的是 parent 在

¹本系統以 LanceDB 的 l2 metric 檢索，其回傳的 `_distance` 對正規化向量而言**已經是平方距離** $d = \|u - v\|_2^2 = 2 - 2\cos(u, v)$ ，故餘弦相似度為 $1 - d/2$ ，不需（也不可）再平方一次。實測 $d = 0.121063$ 時真實餘弦值為 0.9395，與 $1 - d/2$ 精確相符；若誤用 $1 - d^2/2$ 則回報 0.9927，會系統性高估相似度，並使依此挑選的去重門檻實質上遠比宣稱值寬鬆。

活躍層的角色，而非其在圖上的存在——換言之，收縮改變的是技能的可檢索性與層級歸屬，不是節點的去留。此一偏離對 Proposition 1 的影響見 5.5 節 (a)。

Φ 的啟動由三個演化觸發條件控制：任務失敗 (T1)、軌跡長度超過移動平均 (T2)、高共現子序列頻率超過門檻 (T3)；任一條件滿足即啟動演化。

4.5 自我挑戰與課程式自動閉環

為使系統能在無人監督下持續演化，本研究實作課程式自動閉環 (curriculum loop)。系統依代理當前的技能覆蓋分布，自動生成「剛好在能力邊界之外」的前沿任務送入任務佇列。一個同步迴圈交替執行「消費」分支（取出待辦任務並完整跑 $A_1 \rightarrow A_2 \rightarrow A_3$ ）與「生產」分支（佇列清空時自動出題）；每累積固定批量 ($N = 10$) 任務即觸發一次 Φ ，並於演化後刷新 A_2 的圖譜快取，使新插入的技能對後續任務可見，形成 in-process 的完整閉環。

4.6 系統實作與實驗環境

代理執行核心為自建的最小化 ReAct 代理 (SimpleAgent)，以「思考—行動—觀察」迴圈驅動，工具集包含程式碼執行與 shell 執行，均在 Docker 沙盒中隔離執行；網路搜尋工具已實作，但本文的實驗中代理皆未呼叫。語言模型採本地部署的 Gemma-4-26B-A4B（權重全程凍結，不做任何參數更新），運行於單張 RTX 4060 Ti 16GB；技能向量以 LanceDB 儲存與檢索，嵌入模型為 llama-embed-nemotron-8b，於 CPU 執行（常駐約 14 GiB 主記憶體）。各模組以 Python 實作，技能以相容於開放 SKILL.md 標準的格式儲存，兼顧可讀性與工具互通性。

4.7 研究步驟

實作依論文的推理閉環漸進展開：先以開源技能庫完成種子技能匯入（解決冷啟動問題），再依序實作 A_1 、 A_2 、 A_3 三模組，接著串接 Φ 形成完整演化迴圈，最後加入課程式閉環與跨領域驗證。各階段的時程安排與里程碑詳見第 (六) 節之甘特圖。

(五) 預期結果

本系統已完成 A_1 — A_3 三模組、演化算子 Φ 與課程式閉環的實作，並完成一組控制實驗取得初步結果 (5.1)；長期演化與跨領域遷移 (5.2) 則為後續預定執行之驗證。

5.1 初步結果：技能演化的控制實驗

為驗證「改善 A_3 的技能萃取策略能產生更精簡、重用度更高的技能圖譜」，我們設計一組控制實驗：以同一份 50 道程式任務，從完全相同的圖譜初始狀態起跑兩次，唯一變數為 A_3 的反思提示——基準組 (v1) 與改良組 (v2，加入批次內語義抑制，避免重複萃取近義技能)；其餘設定（模型、工具、參數）全部鎖定。主要觀測結果如下表：

觀測指標	結果 (A3-v2 相對 A3-v1)
任務成功率	維持高檔 (49/50 $\rightarrow$ 50/50) ^a
候選技能產出率	-52%
圖譜技能插入數	-45%
平均執行步數	-15% ^b
宏技能浮現	兩組各 1 個 (皆於第一次 Φ 產生)

^a 代理自報之單層 success，當時尚無獨立驗證。 ^b 平均數；兩組中位數皆為 2 步。

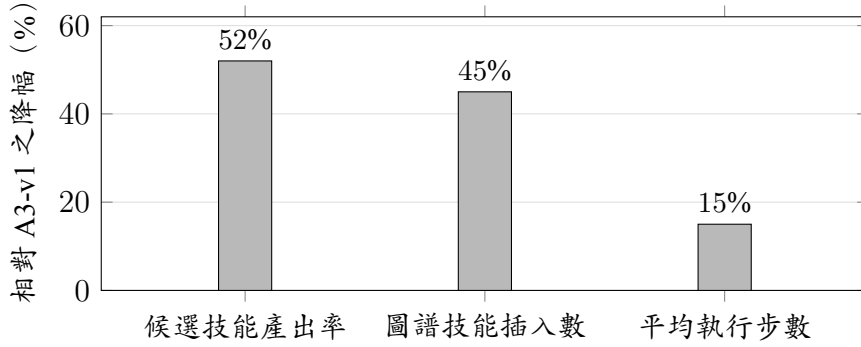


圖 2 A3-v2 相對 A3-v1 之改善幅度（數值越大代表降幅越大、改善越多；平均執行步數為平均數，兩組中位數皆為 2 步）

結果顯示：改良組從源頭抑制了逾半數的冗餘候選技能，圖譜插入數隨之下降 45%，使技能圖譜更為精簡；平均執行步數下降 15%，此為平均數，兩組中位數相同；且為兩種提示詞的單次比較，而非同一系統隨演化的變化，其幅度與 5.4 節量得的逐題雜訊同一量級，故不作為 Proposition 2 的證據。兩組皆透過 Φ -iii 子圖收縮各浮現 1 個宏技能（皆於第一次 Φ 產生），驗證收縮機制確實運作。我們另以五個案例研究（重用、跨群組合、宏技能浮現、跨層檢索、累積效應）質性檢視特定行為，初步觀察到預期的技能重用與組合模式；同時亦發現部分案例的二元判定準則過寬或對應層級不夠精準，此一方法論觀察將用於收緊後續大規模實驗的驗收準則。

5.2 預期結果：長期演化與跨領域遷移

長期演化（百任務課程式 long run）：在課程式閉環驅動下連續執行約 100 個自動出題任務，預期觀測到：(a) 技能庫節點數成長趨於飽和，呼應 Proposition 1（圖譜規模有界）；(b) 技能效用分布的 Gini 係數非遞減，呼應 Proposition 4（高效用技能逐漸集中）；(c) 三層之間的遷移趨於穩定，呼應 Corollary 1（分層收斂）。

跨領域遷移：將於 Python 領域演化成熟之圖譜凍結後，依序在 SQL 查詢與 bash 系統管理兩個新領域各跑約 100 題，觀測軌跡壓縮率 ρ 的變化。預期初期 ρ 因遷移不完美而下降，中後期隨 A_3 萃取出領域專屬技能而回升，作為 SIES 在固定任務分布假設之外的延伸驗證。

5.3 對應之理論命題

上述實驗構成對論文框架核心命題的實證檢驗。茲列出其中數項主要命題（陳述依論文框架，完整假設與證明見論文，此處從略）：

Proposition 1 (圖譜規模有界). 存在常數 K^* ，使技能圖譜的節點數在任意演化時刻有界，特別是活躍層滿足 $|M_{\text{active}}| \leq K^*$ 。

Proposition 2 (規劃深度下降). 隨演化進行，完成同類任務所需的規劃深度（以軌跡長度 $|\tau|$ 衡量）呈下降趨勢。

Proposition 4 (效用分布 Gini 非遞減). 在（近似）穩態任務分布下，技能效用分布的 Gini 係數隨任務累積呈非遞減。

Theorem 1 (結構平衡存在性). 在效用衰減與演化算子 Φ 的反覆作用下，系統存在結構平衡狀態，且演化過程最終收斂至該平衡。

其中 Proposition 2 目前尚無可分辨的實證（見 5.1、5.4 節）；Proposition 1、Proposition 4 與 Theorem 1 則將於第 5.2 節之規模化與跨領域實驗中檢驗。（其餘如 Proposition 3 熵有界收斂、Corollary 1 分層收斂等亦屬同一框架，詳見論文。）

5.4 無技能庫對照實驗

5.1 節的控制實驗僅變動 A_3 的提示詞，兩組皆持有技能庫，因此無法回答「技能庫本身是否帶來效益」。為此本研究另行執行一組無技能庫（no-skill）對照。

設計：對照組與實驗組使用**同一批 50 題凍結題庫**（內容雜湊逐位元相同），於獨立的隔離工作區執行，避免污染既有資料。對照組停用 A_2 的技能檢索與注入，**且不執行演化算子 Φ** ——後者是刻意的設計：若讓技能庫在背景持續成長，而 A_2 又因停用檢索而無法使用它，則比較的對象將變成「有技能庫」對「技能庫在背景演化但用不到」，而非預期的消融對照。此一禁止條件以程式中的具名旗標與專屬迴歸測試固定，並於執行後以「四個週期檢查點皆未產生檔案、圖譜節點與邊數前後相同、演化紀錄檔不存在」三項證據核驗。

結果：兩場的獨立驗證通過率**完全相同**，皆為 $32/43 = 74.4\%$ （分母為具備可用獨立 oracle 的題數；兩場的非獨立驗證題目集合相同，故分母一致、可直接比較）。然而**逐題比對的一致率僅 $33/43 = 76.7\%$** ：有 5 題由成功轉為失敗、另有 5 題由失敗轉為成功，兩個方向各半。換言之，兩個 32 由**不同的題目組成**，總數相同係兩方向翻轉互相抵銷所致，而非技能庫「不影響任何一題」。

解讀：在這份題庫上，**技能庫的效果小於語言模型本身的隨機性**——逐題翻轉幅度達 23% 且方向對稱，與待測效果同一量級。此現象與 5.1 節控制實驗中觀察到的候選技能命名漂移同屬一類問題：相同輸入下的溫度雜訊，其量級與所欲量測的效果相當。成本面則有明確且方向一致的差異：提示詞 token 數下降 19.8%（技能注入消失所致）、耗時中位數下降 15.7%；但 LLM 呼叫次數、軌跡步數與工具呼叫次數的**中位數三者完全相同**，顯示差異來自少數長尾任務而非整體行為改變。 A_3 產出的候選技能由 23 個降為 11 個。

偏誤方向：此題庫係由課程式閉環依**當時的技能庫**自動生成，題目分布貼合該技能庫的能力邊界，最自然的讀法是它**偏向有技能組**。在此偏誤下有技能組仍未勝出，因此此一無效果結果若存在偏誤，方向是使結論**更為保守**，而非誇大。

本結果不支持的宣稱：樣本為 $n = 50$ 、單次執行、無重複測量，本研究不作任何統計顯著性宣稱，亦不宣稱技能庫無效。要區分「效果為零」與「效果被雜訊淹沒」，需要重複測量以估計雜訊水準（見第（七）節主題四）。

5.5 限制條款：目前實作與論文框架之落差

為使上述結果能被正確解讀，以下逐項載明現行實作與論文形式框架之間的落差，以及數項**不應被過度詮釋**的觀測值。

(a) 容量語意的界定——Proposition 1：論文框架的 Definition 10 明訂「系統 enforce 一個硬容量上限 $K \geq K^*$ ，使 $|\Sigma_t| \leq K$ 對所有 t 成立」。現行實作**並未 enforce 全域硬上限**（設定檔中的容量鍵從未被任何程式碼讀取），且如 4.4 節所述， Φ -iii 收縮**不移除 parent**、封存層亦僅停止檢索而**不刪除節點**——實作中不存在任何節點移除路徑（可由演化與代理模組中無任何節點移除呼叫佐證）。

實作中與該上限對應的是**活躍層**：效用衰減僅作用於活躍層技能，未獲強化者在效用跌破下閾值後遷出至冷卻層，因此**活躍層規模由衰減與強化的平衡自然決定**。50 題長跑的實測支持這一點——**總節點數自 90 增至 110，而活躍層始終維持在 30 至 34 之間**。被遷出的技能在冷卻與封存層停止衰減、亦不被移除，故總節點數單調成長，活躍層規模則不隨之擴張。

因此本研究將 Proposition 1 的實證範圍界定為 $|M_{\text{active}}| \leq K^*$ ，而非 $|\Sigma_t| \leq K$ 。5.3 節的命題陳述本即寫為「特別是活躍層滿足 $|M_{\text{active}}| \leq K^*$ 」，故此處為範圍的明確化，而非結論的改口。

此界定的前提須一併載明：它所依賴的是「衰減僅作用於活躍層」這一實作特性，而非論文 Assumption 3（均勻衰減）——後者在本實作中並不成立。全域

容量的 enforce 與節點剪枝機制留作後續工作（見第（七）節）。在此之前，5.2 節「節點數成長趨於飽和」之預期僅適用於活躍層，不應被理解為對總節點數有界的檢驗。

(b) **composes_into 邊為機械產生，不構成學習證據**：每次子圖收縮固定為巨集技能與其兩個 parent 各補一條邊，故此類邊數恆等於收縮次數的兩倍（目前為 2 次收縮、4 條邊）。它所反映的是收縮操作曾被執行，而非「系統學到了技能之間的關聯」。本研究的圖層指標已在「learned edges」的定義中將其排除，論文引用圖連通性數字時亦須同此處理。

(c) **verified 成功率的分母是「具備可用獨立 oracle 的題數」**：2026-09-08 的 50 題長跑中，僅 43 題的驗證規格達到「獨立 oracle」強度 (verified_independent)，其中 32 題通過，故 verified 率應記為 $32/43 = 74\%$ 而非 $32/50$ 。其餘 7 題在該場的實測紀錄為 verified_liveness（僅具存活性檢查強度），且 verified_success 被寫為 False；但此 False 的成因是驗證器在檢查階段拋出 KeyError——那 7 題的規格是既無比對樣式、也無測試案例的裸規格，亦即該題根本不存在可用的 oracle，並非代理未完成任務。驗證器其後已在主程序端將此類規格濾為「無可用驗證」。無論採事後修正或依原始紀錄判讀，正確的分母都是 43。此點若不寫明，讀者會預設 $n = 50$ 而得到 $32/50 = 64\%$ 的錯誤數字；全文引用 verified 數字時一律採 $32/43$ 並明示分母定義。

(d) **provenance 邊的增長並非技能關聯被學到**：目前圖上 216 條邊中，有 206 條屬 follows (91 條) 與 requires (115 條) 兩類 provenance 邊。其權重僅有 0.4 與 0.7 兩個硬編常數，由課程路徑的位置順序機械落地而成。若論文引用「孤島率下降」一類數字，必須同時說明其成因是課程結構被寫入圖中，而非技能之間的關聯被系統學習而得。

(e) **演化觸發條件 T3 目前不具鑑別力**：以既有 324 筆真實軌跡進行離線量測（閾值與判定式未改動，但選樣繞過了程式預設的最近 100 檔上限）顯示，T3（高共現子序列頻率超過門檻）在全部四個切片上皆判定為真；全量語料中超過門檻的 n-gram 為 code_execution → finish (261/324, 80.6%)；若依程式實際的選樣路徑（最近 100 檔、98 份有效軌跡），code_execution → finish (65.3%) 與 bash_execution → code_execution (39.8%) 皆超過門檻。兩者都是 ReAct 迴圈的動作層形狀（前者即標準收尾），與「哪些技能常一起使用」無關。逐題模擬進一步顯示 T3 自第 1 題起即持續觸發、50 題全中（判定式亦無最小軌跡數下限，樣本極小時比例必然衝高）。此一診斷說明：「動作序列的高頻共現」不足以作為「此處值得演化」的判據。4.4 節所列的三個觸發條件中，T3 在現行判定下實質等同於無條件觸發，其判據設計為後續待修項目。另，2026-09-08 的 50 題長跑中 T1—T3 尚未接線，6 次 Φ 均為每 10 題的固定間隔強制執行。

(f) **檢索未依記憶分層過濾**： A_2 的向量檢索不含層級條件 (TD-18 未修)。demo 的 8 組查詢共 48 個 top-6 結果中，冷卻層 24、封存層 15、活躍層 9。因此 (a) 所述的活躍層有界，並不代表檢索候選池有界；三層分級目前只影響效用衰減的作用範圍，以及 Activation Score 中的效用項。

（六）研究流程規劃

本研究依「基礎建置 → 三模組 → 演化算子 → 持續學習整合 → 規模化驗證 → 成果整理」之順序分階段推進。前數個階段已大致完成，後續聚焦於規模化驗證與成果撰寫。各階段重點如下：

階段一 基礎建置與三模組（已完成）：匯入開源種子技能以解決冷啟動，並依序完成任務分解器 A_1 、記憶引導規劃器 A_2 與反思評估器 A_3 ，串成單一任務的推理閉環。

階段二 演化算子與初步驗證（已完成）：接上演化算子 Φ （效用評估、插入、

子圖收縮、三層分級)，並以第（五）節之控制實驗驗證技能層級演化的初步成效。

階段三 持續學習整合（進行中）：加入課程式自動出題的閉環，使系統能在無人監督下持續執行任務、累積並整理技能。

階段四 規模化與跨領域驗證（後續規劃）：進行較大規模的長期執行，並將演化成熟的圖譜遷移至新領域，量測技能重用度與圖譜穩定性的變化（對應第（五）節之預期結果）。

階段五 成果整理（後續規劃）：整理實驗數據與圖表，完成研究成果之撰寫與發表準備。

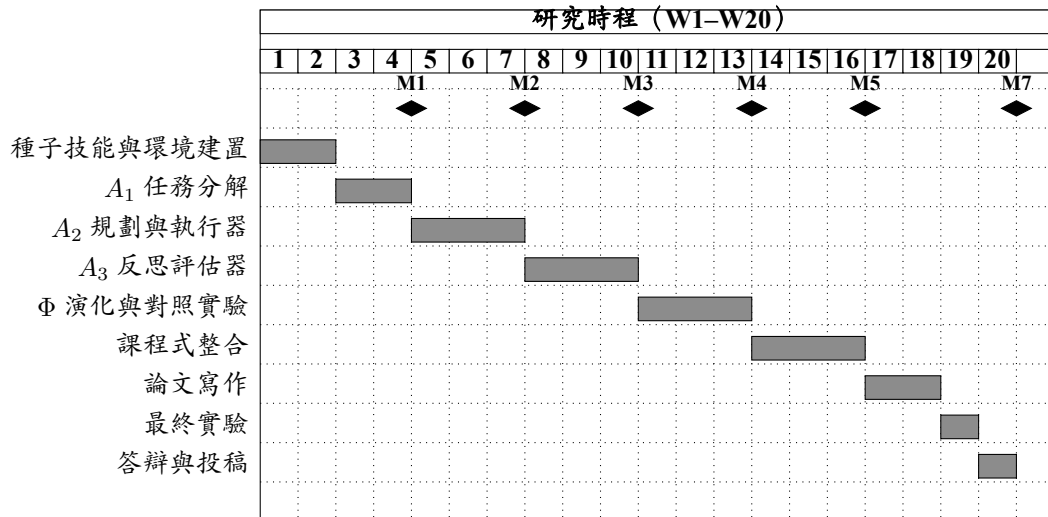


圖 3 研究時程甘特圖（橫條為各階段，菱形為里程碑 M1—M5、M7）

（七）後續工作

本節依據前述實作與量測結果，指出四個尚待證明的主題。各主題均載明「現況為何」「下一步需證明什麼」與「如何驗證」，而非功能清單。

7.1 從檢索到遍歷：圖結構對檢索的實際貢獻

現況：A₂ 的技能選擇是**扁平排序**——以 Activation Score $A(\sigma) = \lambda_1 \cdot \text{sim} + \lambda_2 \cdot U(\sigma) + \lambda_3 \cdot \text{centrality}(\sigma)$ 取前 k 名，圖結構僅以 centrality 作為排序中的一項分項參與，過程中**不做任何鄰居遍歷**。實測顯示在部分查詢上 centrality 分項為 0，原因是 65.5% 的節點為孤島——對這些節點而言，圖結構對其排序毫無作用。

下一步需證明的是：**圖結構是否真的改善了檢索品質**。這不能由「圖上有邊」或「連通度上升」推得，因為既有的邊多數為機械產生（見 5.5 節 (b)、(d)）。驗證方式是加入一跳鄰居擴展，並以「關閉圖結構分項」（令 $\lambda_3 = 0$ 且**不做遍歷**）為對照組，比較兩者的任務成功率。缺少該對照時，無法宣稱圖結構對檢索有貢獻。

7.2 從共現到因果：技能關聯的驗證方式

現況：本研究已實作觀測式關聯偵測（正規化點互資訊搭配單尾 Fisher 精確檢定，並對每族技能對做 Benjamini-Hochberg 偽發現率校正），預設關閉。在真實資料上**尚未產生任何通過門檻的關聯**。

更根本的問題不在門檻是否過嚴，而在於「**一起被引用**」並不等於「**彼此依賴**」：兩個技能同時出現於一次任務，可能只反映該任務的題型，而非其間存在可重用的結構關係。因此下一步需將關聯的驗證改為**介入式**——比較保留、移除或替換某一技能時任務結果的差異，而非統計其共現頻率。此種驗證所需的重複試跑次數，遠高於本研究已執行的規模。

7.3 從工具可用到自主使用

現況：系統已提供按需檢索與閱讀記憶的工具，小樣本觀察到它在需要外部事實時使用該工具、在自足任務（如純算術）時不使用。

然而該使用策略係由 system prompt 提供（提示詞中明載「自足任務應直接作答」一類指示），且每組樣本量為 $n = 1$ ，因此目前的觀察**僅能說明工具鏈可用且未被濫用，不能宣稱模型自主判斷了何時該讀**。下一步需要「有工具」對「無工具」的對照設計，且提示詞中不得含使用策略，並以**任務成功率**而非讀取次數為指標——讀取次數只反映工具被呼叫的頻率，不反映閱讀是否有助於完成任務。

7.4 可分辨的實驗設計

現況：5.4 節的無技能庫對照顯示，在該題庫上技能庫的效果**小於語言模型的隨機性**（逐題翻轉 23% 且方向對稱），單次 $n = 50$ 無法分辨兩者。此外該題庫係由課程式閉環針對當時的技能庫生成，且五個能力群中有兩群（C3、C4）**完全沒有樣本**。

因此在提出任何關於技能庫效益的量化宣稱之前，實驗設計需先具備三個條件：(i) **重複測量**，以估計同一設定下的雜訊水準，作為判定效果是否可分辨的基準；(ii) 一組**與技能庫無關生成的中性題庫**，以消除 5.4 節所述的題庫偏誤；(iii) **涵蓋全部能力群**的題目分布。在這三者到位之前，本研究對技能庫效益的陳述均應維持在描述性層次。

（八）參考文獻

- [1] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Jiaqing Liang, Jinyi Han, Weijia Li, Xinyi Wang, Zhoujia Zhang, Zishang Jiang, Ying Liao, Tingyun Li, Ying Huang, Hao Shen, Hanyu Wu, Fang Guo, Keyi Wang, Zhonghua Hong, Zhiyu Lu, Lipeng Ma, Sihang Jiang, and Yanghua Xiao. GenericAgent: A token-efficient self-evolving LLM agent via contextual information density maximization. *arXiv preprint arXiv:2604.17091*, 2026.
- [3] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for LLM agents. *arXiv preprint arXiv:2601.02553*, 2026.
- [4] Jan Tomášek. Agent Zero: A general-purpose autonomous AI agent framework. <https://github.com/agent0ai/agent-zero>, 2024.
- [5] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [6] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.